Bayesian variable selection for the ACTG data

Ezequiel

2024-01-07

```
# load libraries
library(hdbayes)
library(parallel)
library(matrixStats)
```

Distributions on the model space

Let $\mathcal{M}$ be the set of all possible model and

$$
\begin{aligned}
p(\beta^{(m)} \mid D_0^{(m)}) &= L(\beta^{(m)} \mid D_0^{(m)})\pi_0(\beta^{(m)} \mid d_0), \\
p_0(m) &= \int p(\beta^{(m)} \mid D_0^{(m)}) d\beta^{(m)}, \\
\beta^{(m)} &\sim N(\mathbf{0}, d_0 \times I_p).
\end{aligned}
$$

where $L(\beta^{(m)} \mid D_0^{(m)})$ is the likelihood of the historical data $D_0^{(m)}$ under model m, $\pi_0(\beta^{(m)} \mid d_0)$ is the prior distribution of the regression coefficients, and d_0 is the prior precision of the regression coefficients. We assume that the prior probability of each model is given by

$$
p(m) = \frac{p_0(m)}{\sum_{m \in \mathcal{M}} p_0(m)}.
$$

Now, we set the joint prior distribution for the regression coefficients and a_0 as

$$
\begin{aligned}
\pi(\beta^{(m)}, a_0 \mid D_0^{(m)}) &\propto \frac{L(\beta^{(m)} \mid D_0^{(m)})^{a_0}}{Z(a_0)} a_0^{\delta_0 - 1}(1 - a_0)^{\lambda_0 - 1}\pi_0(\beta^{(m)} \mid c_0), \\
Z(a_0) &= \int L(\beta^{(m)} | D_0^{(m)})^{a_0}\pi_0(\beta^{(m)}) d\beta^{(m)}
\end{aligned}
$$

and the posterior distribution of the model m as

$$
\begin{aligned}
p(m \mid D^{(m)}) &= \frac{p(D^{(m)} \mid m)p(m)}{\sum_{m \in \mathcal{M}} p(D^{(m)} \mid m)p(m)} = \frac{p(D^{(m)} \mid m)p_0(m)}{\sum_{m \in \mathcal{M}} p(D^{(m)} \mid m)p_0(m)}, \\
p(D^{(m)} \mid m) &= \int L(\beta^{(m)} \mid D^{(m)})\pi(\beta^{(m)}, a_0 \mid D_0^{(m)}) d\beta^{(m)} da_0.
\end{aligned}
$$

Computations

First, we scale the data:

```
age_stats <- with(actg036,
                  c('mean' = mean(age), 'sd' = sd(age)))
cd4_stats <- with(actg036,
                  c('mean' = mean(cd4), 'sd' = sd(cd4)))
actg036$age <- ( actg036$age - age_stats['mean'] ) / age_stats['sd']
actg019$age <- ( actg019$age - age_stats['mean'] ) / age_stats['sd']
actg036$cd4 <- ( actg036$cd4 - cd4_stats['mean'] ) / cd4_stats['sd']
actg019$cd4 <- ( actg019$cd4 - cd4_stats['mean'] ) / cd4_stats['sd']

current_data <- actg036
hist_data <- actg019
```

And set the parameters:

```
a0_seq <- seq(0, 1, length.out = 21)
data <- list(current_data, hist_data)
family   <- binomial(link = "logit")
c0_seq <- sqrt(c(3, 5 , 10))
d0 <- 0.5^(1/2)
delta0 <- 25
lambda0 <- 25
iter_warmup <- 1000
iter_sampling <- 2500
```

Now, we create auxiliary functions to compute posterior model probabilities:

```
# function to compute p0
logp0 <- function(formula,
                  ...) {
  hdbayes::glm.npp.lognc(
    formula = formula,
    ...
  )
}

# function to compute Z(a_0)
logncfun <- function(a0,
                     ...){
  hdbayes::glm.npp.lognc(
    a0 = a0,
    ...
  )
}

# function to compute the posterior for regression coefficients
post_beta <- function(formula,
                      c0,
                      data,
                      family,
```

```
                      delta0,
                      lambda0,
                      iter_warmup,
                      iter_sampling) {
  # compute Z(a_0) for each value of a_0
  a0.lognc <- lapply(a0_seq,
                     logncfun,
                     formula = formula,
                     family = family,
                     histdata = data[[2]],
                     iter_warmup = iter_warmup,
                     iter_sampling = iter_sampling,
                     chains = 4,
                     refresh = 0,
                     beta.sd = c0
  )
  a0.lognc <- data.frame(do.call(rbind, a0.lognc))
  # compute the posterior for the regression coefficients
  fit <- hdbayes::glm.npp(formula,
                          data = data,
                          family = family,
                          a0.lognc = a0.lognc$a0,
                          lognc = matrix(a0.lognc$lognc, ncol = 1),
                          a0.shape1 = delta0,
                          a0.shape2 = lambda0,
                          iter_warmup = iter_warmup,
                          iter_sampling = iter_sampling,
                          chains = 4,
                          refresh = 0,
                          beta.sd = c0
  )
}

# function to compute the posterior model probabilities
post_models <- function(logp0_models, logml_models) {
  logp0_models <- sapply(logp0_models, function(x) x[2])
  logml_models <- unlist(data.frame(do.call(rbind, logml_models))$logml)
  post_model <- exp(logml_models + logp0_models -
                      logSumExp(logml_models + logp0_models))
}
```

Lastly, we create a function to compute the results:

```
# function to compute the results
samples_models <- function(data,
                           outcome,
                           family,
                           a0_seq,
                           c0,
                           d0,
                           delta0,
                           lambda0,
                           iter_warmup,
                           iter_sampling,
```

```
                        num_cores = 10) {
# get data
current_data <- data[[1]]
hist_data <- data[[2]]

# create list of formulas
covariates <- colnames(current_data)[colnames(current_data) != outcome]
pset <- powerset(covariates)
formulas <- lapply(pset, create_formula, outcome = outcome)

# get covariates from models
covariates_models <- lapply(pset, get_covariates)

# define cluster
cl <- parallel::makeCluster(num_cores)
on.exit(parallel::stopCluster(cl))
parallel::clusterExport(cl, varlist = c('a0_seq',
                                        'family',
                                        'data',
                                        'c0',
                                        'd0',
                                        'delta0',
                                        'lambda0',
                                        'iter_warmup',
                                        'iter_sampling',
                                        'logncfun'),
                        envir = environment()
)
# compute p0
logp0_models <- parLapply(
  cl = cl,
  X = formulas,
  fun = logp0,
  family = family,
  histdata = hist_data,
  a0 = 1,
  iter_warmup = iter_warmup,
  iter_sampling = iter_sampling,
  chains = 4,
  refresh = 0,
  beta.sd = d0
)

# compute posterior for regression coefficients
post_betam  <- parLapply(
  cl = cl,
  X = formulas,
  fun = post_beta,
  c0 = c0,
  data = data,
  family = family,
  delta0 = delta0,
  lambda0 = lambda0,
```

```
    iter_warmup = iter_warmup,
    iter_sampling = iter_sampling
  )

  # compute logml for models
  logml_models <- parLapply(
    cl = cl,
    X = post_betam,
    fun = hdbayes::glm.logml.npp
  )

  # compute prior model probabilities
  prior_models <- exp(sapply(logp0_models, function(x) x[2]) -
                        logSumExp(sapply(logp0_models, function(x) x[2])))
  )

  # compute posterior model probabilities
  post_models <- post_models(logp0_models, logml_models)

  # create data frame with results
  df_post <- data.frame(model = unlist(covariates_models),
                        prior_model = prior_models,
                        ml = exp(unlist(data.frame(do.call(rbind, logml_models))$logml)),
                        post_model = post_models
  )

  # order data frame
  df_post_ord <- df_post[order(df_post$post_model, decreasing = TRUE),]
  rownames(df_post_ord) <- 1:nrow(df_post_ord)
  # return results
  return(list(logp0_models = logp0_models,
              post_betam = post_betam,
              logml_models = logml_models,
              df_post = df_post,
              df_post_ord = df_post_ord))
}
```

Now, we run the model for each value of c_0:

```
post_samples_c01 <- samples_models(data = data,
                                   outcome = "outcome",
                                   family = family,
                                   a0_seq = a0_seq,
                                   c0 = c0_seq[1],
                                   d0 = d0,
                                   delta0 = delta0,
                                   lambda0 = lambda0,
                                   iter_warmup = iter_warmup,
                                   iter_sampling = iter_sampling,
                                   num_cores = 10)
post_samples_c02 <- samples_models(data = data,
                                   outcome = "outcome",
                                   family = family,
```

```
                                    a0_seq = a0_seq,
                                    c0 = c0_seq[2],
                                    d0 = d0,
                                    delta0 = delta0,
                                    lambda0 = lambda0,
                                    iter_warmup = iter_warmup,
                                    iter_sampling = iter_sampling,
                                    num_cores = 10)
post_samples_c03 <- samples_models(data = data,
                                    outcome = "outcome",
                                    family = family,
                                    a0_seq = a0_seq,
                                    c0 = c0_seq[3],
                                    d0 = d0,
                                    delta0 = delta0,
                                    lambda0 = lambda0,
                                    iter_warmup = iter_warmup,
                                    iter_sampling = iter_sampling,
                                    num_cores = 10)
```

Results

The table below shows the posterior model probabilities for each model and each value of c_0.

c0	Model	$p(m)$	$p(D^{(m)} \mid m)$	$p(m \mid D^{(m)})$
3	(age, treatment, cd4)	$4.348783e-01$	$6.257397e-16$	$4.433473e-01$
5	(age, treatment, cd4)	$4.355241e-01$	$6.504253e-16$	$4.509770e-01$
10	(age, treatment, cd4)	$4.354956e-01$	$6.682954e-16$	$4.653851e-01$